CONCLUSIONES — TP02 Programación I

Grupo: 3
Comisión: .....
Robot utilizado: Unitree Go2 (modo simulado con MuJoCo)

a) ¿Cuál fue la principal dificultad de instalación?

La principal dificultad de instalación fue instalar todas las herramientas necesarias para el TP, teniendo que verificar cada paso antes de avanzar.

b) ¿Qué error requirió mayor investigación?

El Problema N.° 5: después de implementar las cuatro funciones, el programa seguía mostrando solo "Conectado / Desconectado", igual que la plantilla. Primero sospeché de la lógica del código, y recién al intentar abrir el archivo desde la carpeta equivocada me di cuenta de que el código nunca se había guardado en mi_desarrollo\mi_tp02.py.

c) ¿Qué solución resultó efectiva y por qué?

Usar rutas absolutas. Resolvió el Problema N.° 3 (cd ..) y fue la clave del Problema N.° 5 (abrir y guardar el archivo correcto). Funciona porque no depende de en qué carpeta esté parada la terminal: la ruta completa siempre apunta al mismo lugar, mientras que una ruta relativa como cd mi_desarrollo solo funciona desde una ubicación específica.

d) ¿Qué intento no funcionó y qué aprendí de él?

Repetir cd .. para "seguir buscando" la carpeta del proyecto no funcionó: la terminal ya estaba en la raíz del disco (C:\) y no había más niveles hacia arriba. Aprendí que cd .. sube un nivel desde la ubicación actual, y no funciona como un botón de "atrás" que recuerda las carpetas visitadas.

e) ¿Qué diferencia encontré entre una guía antigua y la arquitectura actual?

La consigna en PDF usaba un simulador de logs (unitree_sdk_sim.py) y expresaba los movimientos como distancia/velocidad y ángulo/sentido ("izquierda", "derecha"). La arquitectura actual conecta el programa con un gemelo digital en MuJoCo a través de robot.py. Los movimientos se expresan como velocidad y tiempo, y el sentido del giro lo da el signo de la velocidad. Además, los límites de seguridad ya no son constantes que defino yo: los controla el simulador (0.2 m/s, 0.5 rad/s, 10 s por orden) y los informa con ErrorDeSeguridad. Por eso el programa tiene dos niveles de rechazo: la validación de formato propia y el rechazo del robot.

f) ¿Hasta qué nivel logré avanzar?

Hasta el **Nivel 10 — Reporte final generado**, el máximo de la escala. Ejecuté cuatro misiones contra el gemelo digital (MISION_BASICA, MISION_CON_ERRORES, MISION_CUADRADO y una misión propia), cada una con su reporte de comandos ejecutados y rechazados, distancia recorrida y duración estimada.

g) Si tuviera que continuar trabajando, ¿cuál sería el próximo paso?

Leer las misiones desde un archivo de texto en lugar de escribirlas en el código (la Extensión 1 de la consigna), para poder cambiar de misión sin abrir el programa. También agregaría un menú usando el diccionario MISIONES de misiones.py, para elegir qué misión correr, y probaría MISION_LARGA.

Opinión personal

Lo que más me sirvió de este TP fue darme cuenta de que el problema de programación en sí (implementar las funciones) fue la parte más corta del trabajo. La mayor parte del tiempo se fue en instalar y configurar el entorno, y en entender por qué algo que "debería funcionar" no funcionaba. Me quedó grabado el error de no guardar el código en el archivo correcto: por un momento pensé que había algo mal en mi lógica, cuando el problema era mucho más básico. Eso me hizo más cuidadoso a la hora de verificar en qué carpeta estoy parado antes de asumir que algo salió mal en el código.

Si tuviera que rehacer el TP, empezaría verificando el entorno completo (Python, pip, Git) antes de tocar cualquier otra cosa, para no descubrir a mitad de camino que falta una herramienta básica.